



Regularizations

Deep Neural Networks
Session 10
Pramod Sharma
pramod.sharma@prasami.com

2 Agenda

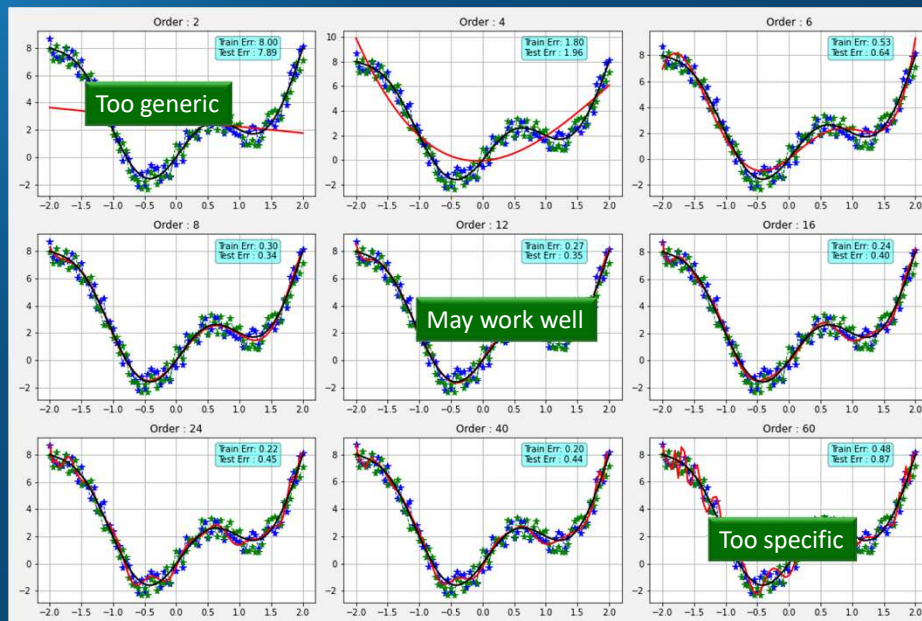
- L-1, L-2
- Dropout
- Early Stopping
- Augmentation

5/8/2026

pra-sami

3

Under-fitting vs. Over-fitting



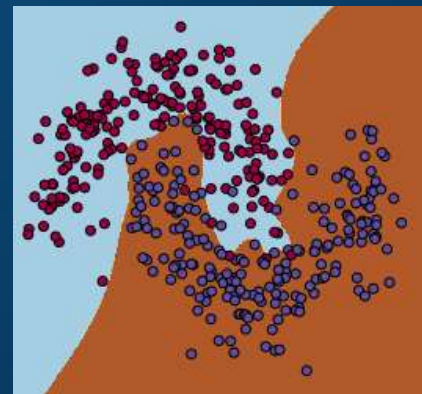
5/8/2026

pra-sami

4

Regularization

- ❑ Regularization helps in avoiding over fitting by penalizing the coefficients
- ❑ In deep learning, it actually penalizes the weight matrices of the nodes
- ❑ Different Regularization Techniques in Deep Learning
 - ❖ L1 regularization
 - ❖ L2 regularizations
 - ❖ Dropout
 - ❖ Early stopping
 - ❖ Data Augmentation
 - MixUp / CutMix – Advanced data augmentation techniques
 - ❖ Batch Normalization(often debated)
 - ❖ Weight Decay for adaptive optimizers
 - ❖ Stochastic Depth – For very deep networks (ResNet)
 - ❖ Weight Initialization - Proper init prevents need for regularization
 - ❖ Label Smoothing – Popular for Transformers/LLMs (not covered here)



5/8/2026

pra-sami

5

Weights vs. Bias

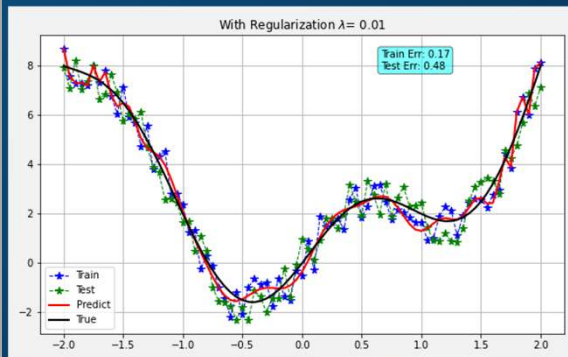
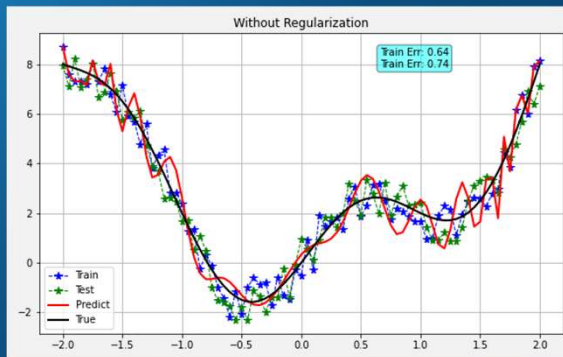
- For neural networks, we typically choose to use a parameter norm penalty λ
 - ❖ Penalizes only the weights at each layer and leaves the biases un-regularized.
- The biases typically require less data to fit accurately than the weights.
- Fitting the weight will requires observing both (fan_in and fan_out) layer in a variety of conditions.
- Each bias controls only a single layer.
 - ❖ This means that we do not induce too much variance by leaving the biases un-regularized.
 - ❖ Regularizing the bias parameters can introduce a significant amount of under fitting.

5/8/2026

pra-sami

6

Effect of L2 Regularization



5/8/2026

pra-sami

7

Theory – Logistic Regression – L1 & L2

- Idea is to minimize Cost Function

$$\diamond J(W, b) = \frac{1}{m} * \sum \ell(a, y)$$

$$\diamond = -\frac{1}{m} \sum \{y * \log(a) + (1-y) * \log(1-a)\}$$

- A term is added to Cost function $\frac{\lambda}{2 * m} * \|W\|_2^2$

$$J(W, b) = \frac{1}{m} * \sum \ell(a, y) + \frac{\lambda}{2 * m} * \|W\|_2^2$$

5/8/2026

pra-sami

8

Theory – Logistic Regression – L1 & L2

$$\square J(W, b) = \frac{1}{m} * \sum \ell(a, y) + \frac{\lambda}{2 * m} * \|W\|_2^2 + \frac{\lambda}{2 * m} * b^2$$

- ✦ This is referred as L2 regularization
- ✦ Regularization Hyperparameter λ : It is another parameter we tune...

$$\square \|W\|_2^2 = \sum_{j=1}^n w_j^2 = W^T \cdot W$$

- Here, we are using Euclidean Norm or L2 Norm

- Compared to W, bias b has fewer dimensions, hence, it is generally not considered

- If you add for b, $(\frac{\lambda}{2 * m} * b^2)$... that may help

- ✦ Although its effect will be minimal,
- ✦ Better to leave it alone.

5/8/2026

pra-sami

9

Theory – Logistic Regression – L1 & L2

□ Sometimes L1 too is used

$$\square J(W, b) = \frac{1}{m} * (\sum \ell(a, y)) + \frac{\lambda}{m} \cdot \|W\|_1$$

$$\square \text{Differentiation of } \frac{\lambda}{m} \cdot \|W\|_1 = \frac{\lambda}{m} \text{ sign}(W)$$

Magnitude of L2 Decreases as W get smaller

Constant magnitude force pushing weights toward zero, unlike.

	L2 Regularization	L1 Regularization
Penalty term	$(\lambda/2m) \cdot \sum W_j^2$	$(\lambda/m) \cdot \sum W_j $
Also called	Ridge, Weight Decay	Lasso
Effect on weights	Shrinks toward 0 (not exactly 0)	Drives weights to exactly 0
Result	Dense, small weights	Sparse weights
Use case	General deep learning	Feature selection

5/8/2026

pra-sami

10

Neural Network – Frobenius Norm

□ The Challenge: Multiple Layers

❖ In neural network, we have different layers with different weights

□ So we look at its cumulative effect over all layers

□ Hence the Cost function

$$\diamond J(W, b) = J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]}, \dots, W^{[L]}, b^{[L]})$$

$$\diamond J_{reg}(W, b) = -\frac{1}{m} * \sum \{y * \log(a) + (1-y) * \log(1-a)\} + \frac{\lambda}{2 * m} \cdot \sum_{l=1}^L \|W^{[l]}\|_F^2$$

$$\diamond \text{Where for any Layer } l \quad \|W^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$$

➢ W is $(n^{[l-1]}, n^{[l]})$ dimensional matrix

➢ The Frobenius norm is the matrix analog of the Euclidean (L2) norm for vectors.

□ It is called *Frobenius norm* of a matrix

❖ *Frobenius norm* defined as the square root of the sum of the absolute squares of its elements

5/8/2026

pra-sami

11

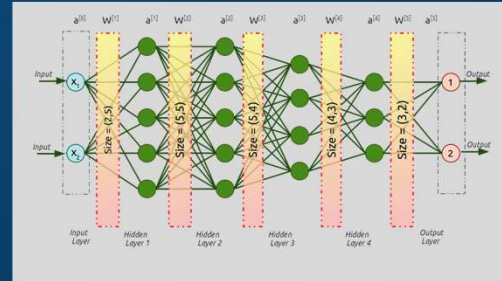
Frobenius Norm of a Vector

$$\square \|A\|_F = \sqrt{\sum (a_{ij})^2}$$

$$\square \text{ i.e. } \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \sqrt{(1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 + 8^2 + 9^2)}$$

□ Layer 1: $W^{[1]}$ has shape (2, 5)

$$\diamond \|W^{[1]}\|_F^2 = \sum_{i=1}^2 \sum_{j=1}^5 (w_{ij}^{[1]})^2$$



5/8/2026

pra-sami

12

Updates to weights

□ Earlier

$$\diamond \partial W^{[l]} = \frac{1}{m} (a^{[l-1]})^T \cdot \partial z^{[l]}$$

$$\diamond \text{ And } W^{[l]} = W^{[l]} - \alpha \cdot \partial W^{[l]}$$

□ For Regularization we add an extra term at the end

$$\diamond \partial W^{[l]} = \frac{1}{m} (a^{[l-1]})^T \cdot \partial z^{[l]} + \frac{\lambda}{m} \cdot W^{[l]}$$

$$\diamond W^{[l]} = W^{[l]} - \alpha \cdot \left[\frac{1}{m} (a^{[l-1]})^T \cdot \partial z^{[l]} + \frac{\lambda}{m} \cdot W^{[l]} \right]$$

$$\diamond W^{[l]} = \left(1 - \frac{\alpha \cdot \lambda}{m} \right) \cdot W^{[l]} - \alpha \cdot \left(\frac{1}{m} (a^{[l-1]})^T \cdot \partial z^{[l]} \right)$$

Weight Decay

Mathematically, we can show that it is still a valid definition of $\partial W^{[l]}$

5/8/2026

pra-sami

13

When to Use L1 or L2 or Both

L1 Regularization (Lasso)	L2 Regularization (Ridge)	Elastic Net (L1 + L2)
- Use for feature selection - L1 tends to push some weights to exactly zero, effectively removing irrelevant features - You suspect many features are irrelevant (i.e., the problem is sparse). - Interpretability is important - Easier to explain with fewer variables.	- Use to reduce model complexity without eliminating features. - All features are useful, but you want to avoid overfitting. - You need a stable solution - Less sensitive to small data changes	If you're unsure which to choose or want both sparsity and stability

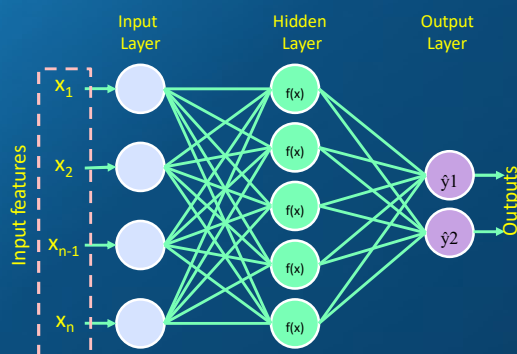
5/8/2026

pra-samī

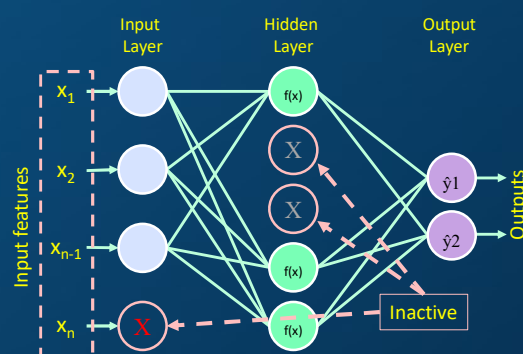
14

Regularization : Dropout

Original



With Dropout



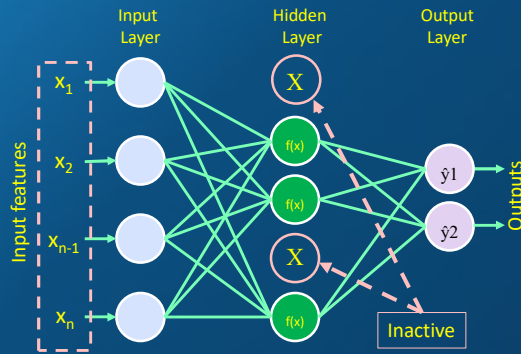
5/8/2026

pra-samī

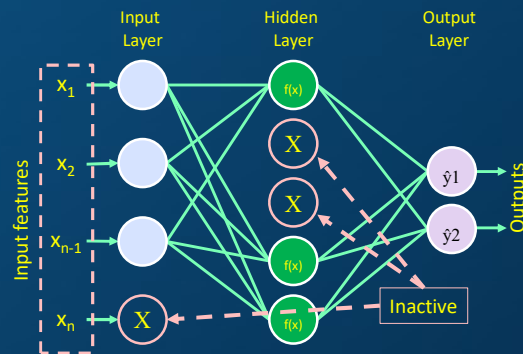
15

Regularization : Dropout

Iteration 1



Iteration 2



5/8/2026

pra-sami

16

Why Dropout Works

Dropout prevents neurons from becoming too specialized and dependent on each other.

Ensemble effect

❖ Training many different subnetworks implicitly

Prevents co-adaptation

❖ Neurons cannot rely on specific other neurons

Forces redundancy

❖ Network learns multiple independent features

Regularization

❖ Adds noise to activations

Layer Type	Typical Dropout Rate (p)	Notes
Input layer	0 - 0.2	Low rate (don't drop too many inputs)
Hidden layers	0.3 - 0.5	Standard range
Output layer	0.0	Never dropout output
Convolutional layers	0.1 - 0.3	CNNs need less dropout
Recurrent layers	0.2 - 0.3	Special variants (variational dropout)

5/8/2026

pra-sami

17

Regularization : Early Stopping

- ❑ How long to train the model?
- ❑ Duration of training → under – fit or over – fit
- ❑ Train the model to the point where its performance on test set is best!
- ❑ Very simple and very effective

How:

- ❑ Train the model and monitor performance
- ❑ Save weight every time the performance improves
- ❑ Stop training if performance has not improved for N epochs
- ❑ It's the last parameter to tune
 - ❖ Repeated early stopping may lead to over-fitting the validation set
 - ❖ Example : K-fold

Train the model until its performance on the validation set stops improving — then stop.

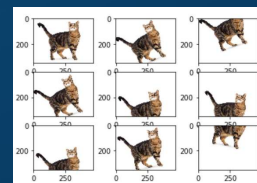
5/8/2026

pra-sâmi

18

Regularization : Data Augmentation

- ❑ Deep learning models are data-hungry. But in most real-world scenarios, limited data is the norm, not the exception.
- ❑ Very effective in image identification
 - ❖ Can be used for other domains as well
- ❑ Most libraries have Image Generators (parameter driven)
 - ❖ Horizontal and Vertical Shift and many more
- ❑ Data augmentation is the most effective regularization technique for computer vision.
- ❑ Test-Time Augmentation (TTA)
 - ❖ Apply augmentations during inference and average predictions



<https://towardsdatascience.com/image-augmentation-for-deep-learning-histogram-equalization-a71387f609b2>

5/8/2026

pra-sâmi

19

Augmentation for Different Domains

Domain	Common Augmentations
Images	Flip, rotate, crop, zoom, color jitter, noise
Text	Synonym replacement, back-translation, random swap
Audio	Time stretching, pitch shifting, noise addition
Time Series	Time warping, scaling, masking
Tabular Data	SMOTE, random noise, feature dropout

5/8/2026

pra-sami

20

Common Image Augmentations

Augmentation	What It Does	Typical Range	Invariance Learned
Horizontal Flip	Mirror left-right	$p=0.5$	Left-right orientation
Vertical Flip	Mirror top-bottom	$p=0.5$ (rare)	Up-down orientation
Random Rotation	Rotate image	$\pm 15^\circ$ to $\pm 30^\circ$	Camera tilt
Horizontal/Vertical Shift	Translate image	± 10 -20%	Object position
Random Zoom	Scale image	0.8-1.2×	Object distance
Brightness/Contrast	Adjust color	$\pm 20\%$	Lighting conditions
Random Noise	Add Gaussian noise	$\sigma=0.01$ -0.05	Sensor noise
Shear	Skew image	$\pm 15^\circ$	Perspective distortion

5/8/2026

pra-sami

21

Advanced Augmentation Techniques

Technique	Description	Best For
MixUp	Linear interpolation of two images + labels	Vision, classification
CutMix	Cut patch from one image, paste into another	Vision
RandAugment	Automatically searches for best augmentation policy	Vision
AutoAugment	Learned augmentation policies (Google)	Vision
Back-Translation	Translate text to another language and back	NLP

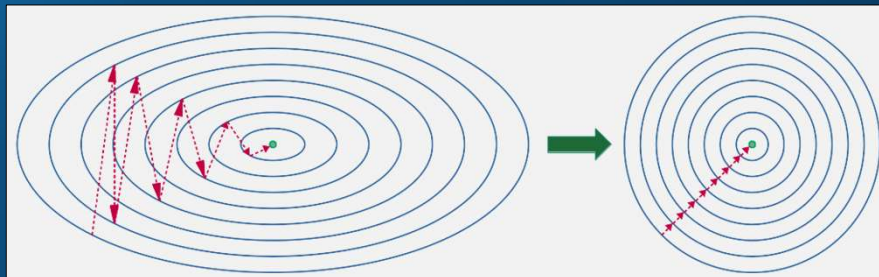
5/8/2026

pra-sâmi

22

Normalization

- ❑ It definitely helps to normalize input data
- ❑ Gradient converges faster
 - ❖ It stabilizes and accelerates the training process



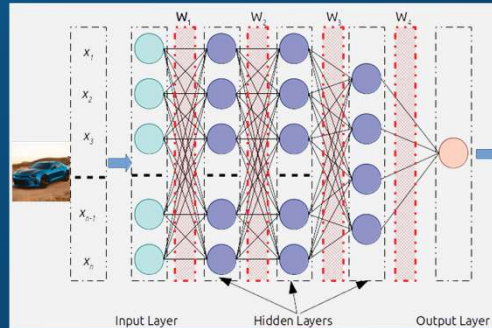
5/8/2026

pra-sâmi

23

Batch Normalization

- ❑ What about hidden layer?
- ❑ After all activations from previous layer are inputs for current layer...



- ❑ Will it help if we normalize the hidden layers too?

5/8/2026

pra-sami

24

Batch Normalization [Sergey Lofte and Christian Szegedy, 2015 - ICML]

- ❑ Batch normalization (also known as batch norm)
- ❑ Make deep networks faster and more stable by normalizing internal representations
- ❑ Benefits
 - ❖ Faster convergence – enables higher learning rates
 - ❖ More stable – reduces sensitivity to initialization
 - ❖ Wider hyperparameter choices – less tuning needed
 - ❖ Mild regularization – reduces need for Dropout in some cases

5/8/2026

pra-sami

25

Batch Normalization

- In theory, its normalizing activation values of the respective layers

- ❖ In practice, it works better if we normalize 'z'

- In General, any Z^i can be normalized

$$Z_{Norm}^i = \frac{Z^i - \mu}{\sqrt{\sigma^2}}$$

$$\begin{aligned} \text{mean } \mu &= \frac{\sum Z^i}{m} \\ \text{std } \sigma^2 &= \frac{1}{m} \sum (Z^i - \mu)^2 \end{aligned}$$

5/8/2026

pra-sâmi

26

Batch Normalization

- In theory, its normalizing activation values of the respective layers

- ❖ In practice, it works better if we normalize 'z'

- In General, any Z^i can be normalized

$$Z_{Norm}^i = \frac{Z^i - \mu}{\sqrt{\sigma^2}}$$

$$\hat{z} = \gamma \cdot Z_{Norm}^i + \beta$$

- ❖ where γ and β parameters we can train (network decides optimal scale/shift)

- if $\gamma = \frac{1}{\sqrt{\sigma^2}}$ and $\beta = \frac{\mu}{\sqrt{\sigma^2}}$; $Z_{Norm}^i = \hat{z}$

Instead of using z_{Norm}^i , researchers realized that it's better to derive $\hat{z}$ with two trainable parameters.

Intuition is that by normalizing z , we are introducing bias in the system. Hence it makes sense to let network decide them

5/8/2026

pra-sâmi

27

Batch Normalization

- In theory, its normalizing activation values of the respective layers

- ❖ In practice, it works better if we normalize 'z'

- In General, any Z^i can be normalized

$$Z_{Norm}^i = \frac{Z^i - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\hat{z} = \gamma \cdot Z_{Norm}^i + \beta$$

- ❖ where γ and β parameters we can train (network decides optimal scale/shift)

- if $\gamma = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}$ and $\beta = -\frac{\mu}{\sqrt{\sigma^2 + \varepsilon}}$; $Z^i_{Norm} = \hat{z}$

Let's add a small ε to prevent zero divide error...

5/8/2026

pra-sâmi

28

Batch Normalization

- Training:

- ❖ Batch norm is used along with mini batches
 - ❖ Batch norm is applied to the batch under consideration only irrespective of other mini batches
 - Mini-batch B_1 : $\mu_1, \sigma_1^2 \rightarrow$ normalize B_1
 - Mini-batch B_2 : $\mu_2, \sigma_2^2 \rightarrow$ normalize B_2
 - Mini-batch B_3 : $\mu_3, \sigma_3^2 \rightarrow$ normalize B_3

- Testing (inference)

- ❖ Batch norm uses running averages accumulated across training batches — not the current batch's statistics

- Why Per-Batch, Not Global?

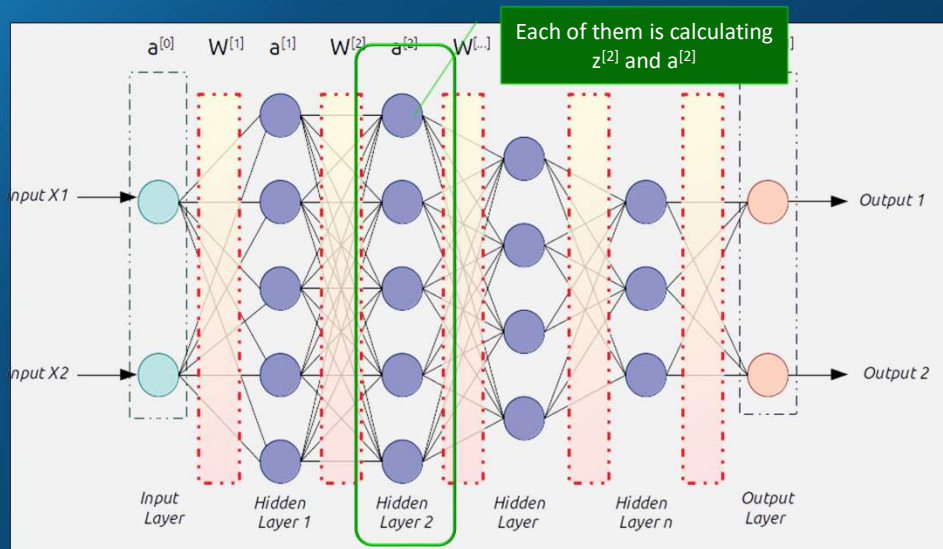
	Per-Batch (BN)	Global (Dataset)
Computation	Cheap, online	Expensive, requires full pass
Stochasticity	Adds regularization (noise)	Deterministic
Distribution shift	Adapts naturally	Fixed

5/8/2026

pra-sâmi

29

Where does it fit in overall scheme?

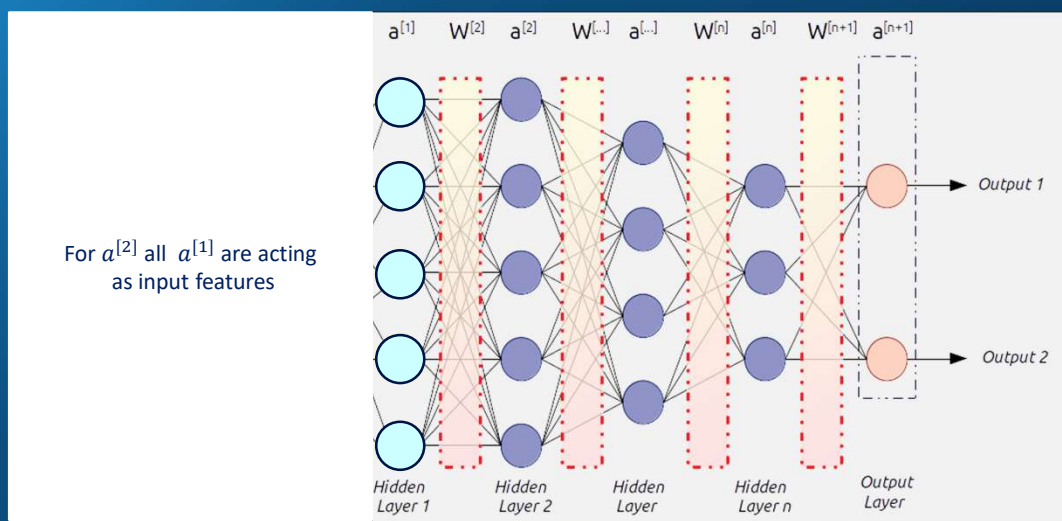


5/8/2026

pra-sami

30

Where does it fit in overall scheme?



5/8/2026

pra-sami

31

Covariance Shift

- Covariate shift: the distribution of input features changes between training and testing
- Internal covariate shift: Layer inputs change during training due to prior layer updates
- Example:

Phase	Input Distribution	Model Sees
Training	Heights: 150-180 cm	Mostly adult heights
Testing	Heights: 80-120 cm	Only children's heights

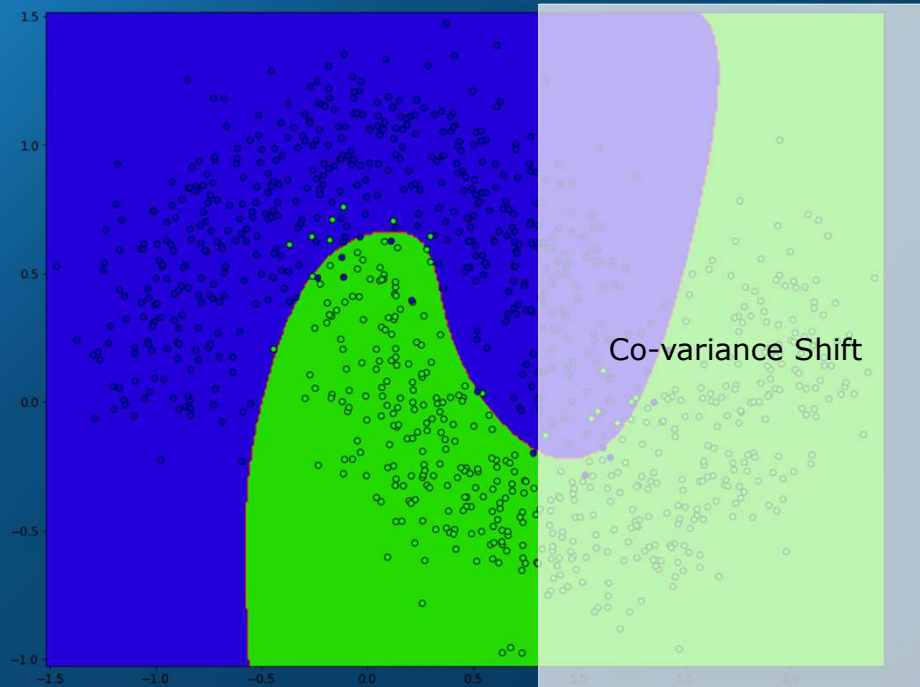
- Internal covariate shift happens inside a neural network
 - ❖ Each layer's input distribution keeps shifting as earlier layers update

Internal covariate shift = when a layer's input distribution keeps changing because earlier layers are still learning. Batch Norm fixes this by normalizing each layer's input to a stable distribution.

5/8/2026

pra-sami

32



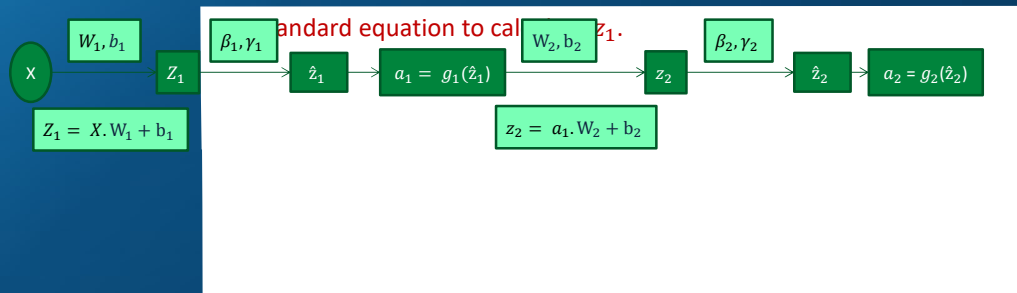
5/8/2026

pra-sami

33

Batch Normalization

- Forward and back propagation with batch norm:



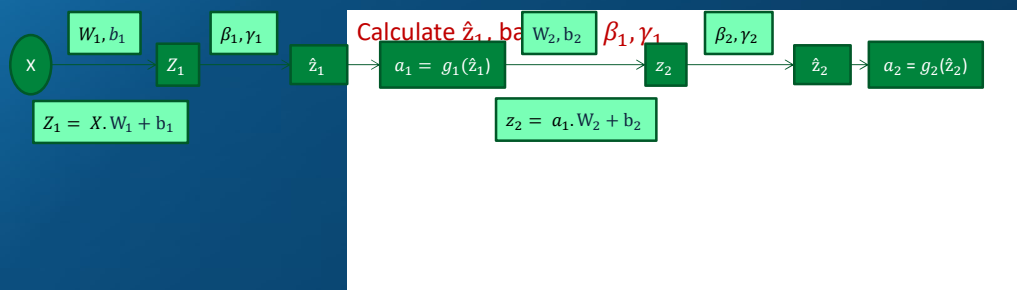
5/8/2026

pra-sami

34

Batch Normalization

- Forward and back propagation with batch norm:



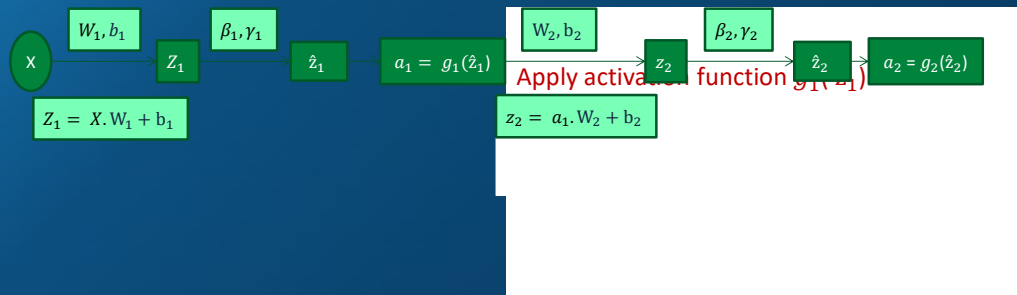
5/8/2026

pra-sami

35

Batch Normalization

- Forward and back propagation with batch norm:



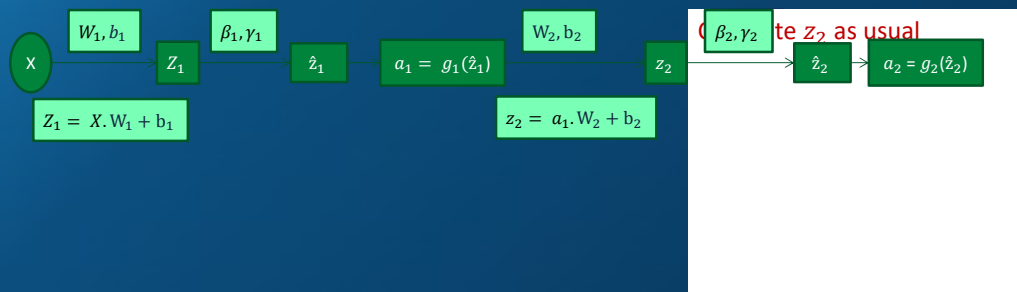
5/8/2026

pra-sami

36

Batch Normalization

- Forward and back propagation with batch norm:



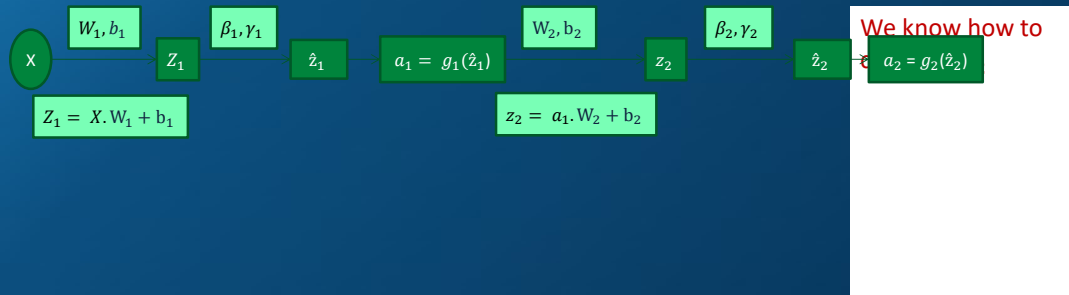
5/8/2026

pra-sami

37

Batch Normalization

- Forward and back propagation with batch norm:



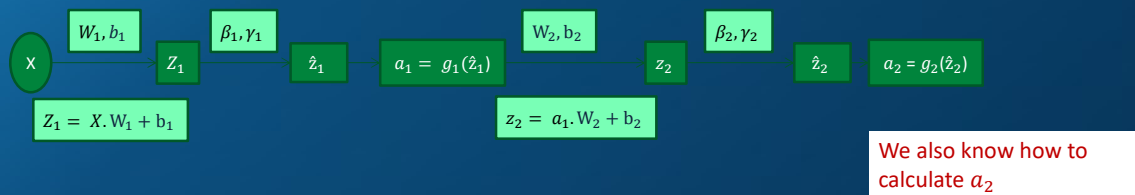
5/8/2026

pra-sami

38

Batch Normalization

- Forward and back propagation with batch norm:



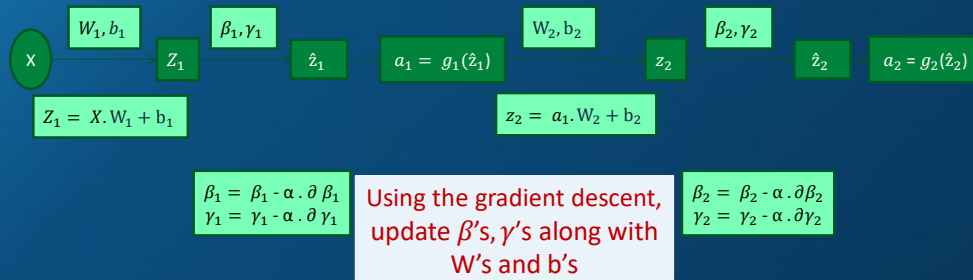
5/8/2026

pra-sami

39

Batch Normalization

□ Forward and back propagation with batch norm:



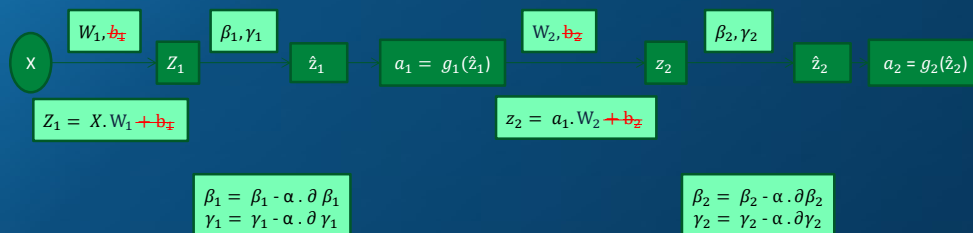
5/8/2026

pra-sami

40

Batch Normalization

□ Forward and back propagation with batch norm:



One more thing, since we are normalizing our Z 's, keeping b 's in the equation does not make any sense now.
Being a constant, it will get eliminated!!

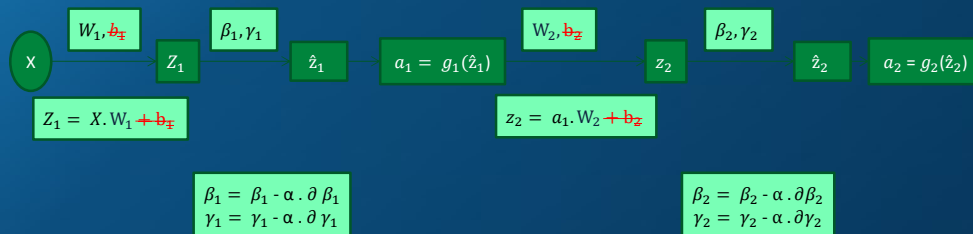
5/8/2026

pra-sami

41

Batch Normalization

- Forward and back propagation with batch norm:



And at test/validation time, use an exponentially weighted average!
 So while training do not forget to save exponentially weighted values or simply running average!!

5/8/2026

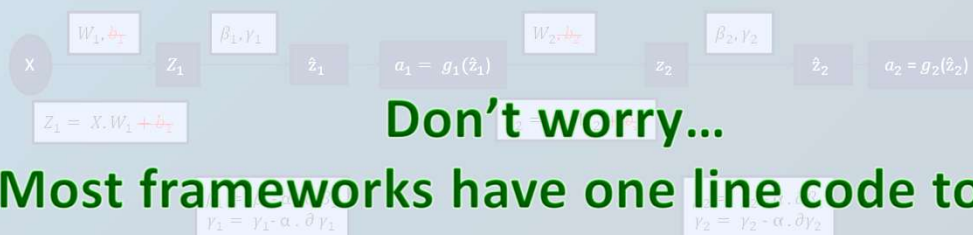
pra-sami

42

Batch Normalization

- Forward and back propagation with batch norm:

Too many calculation steps...



Don't worry...

Most frameworks have one line code to do it.

5/8/2026

pra-sami

43

Batch Normalization – Code Sample

```
model = tf.keras.models.Sequential(
    [
        tf.keras.layers.RNN( keras.layers.LSTMCell(units), input_shape=(None, input_dim) ),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Dense(output_size),
    ]
)
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.dense1 = nn.Linear(in_features=320, out_features=50)
        self.dense1_bn = nn.BatchNorm1d(50)
        self.dense2 = nn.Linear(50, 10)
```

- ❑ And it is applied to mini batches only....
- ❑ Batch Norm can be updated using any of the optimization functions...

5/8/2026

pra-sâmi

44

Batch Normalization



Remember β, γ are parameters you train!

5/8/2026

pra-sâmi

45

Layer Normalization

- ❑ Normalizes activations across the features/channels (instead of batch).
- ❑ Applied per-instance, making it independent of batch size.
- ❑ Fixes internal covariate shift (like BatchNorm) but works for:
 - ❖ Small or variable batch sizes (e.g., RNNs, Transformers).
 - ❖ Recurrent architectures (unlike BatchNorm).
- ❑ Improves training stability and convergence.
- ❑ BatchNorm: Normalizes across batch dimension. (torch.nn. BatchNorm)
- ❑ LayerNorm: Normalizes across feature dimension. (torch.nn.LayerNorm)

5/8/2026

pra-sâmi

46

Weight Decay for Adaptive Optimizers

- ❑ Weight Decay (L2) and Adaptive Optimizers (Adam, RMSprop, etc.)
 - ❖ When you add weight decay to Adam the naive way, the weight decay term gets divided by the adaptive learning rate too!
- ❑ Problem: The weight decay effect is not constant
 - ❖ Depends on $\hat{v}_t$ (gradient history)
 - ❖ Works poorly in practice
- ❑ Parameter with large gradient history (high $\hat{v}_t$):
 - ❖ Weight decay effect = $\frac{\eta\lambda}{large} \approx \text{VERY SMALL}$
 - ❖ Parameter doesn't decay properly
- ❑ Parameter with small gradient history (low $\hat{v}_t$):
 - ❖ Weight decay effect = $\frac{\eta\lambda}{small+\epsilon} \approx \text{VERY LARGE}$
 - ❖ Parameter decays too aggressively

The Solution: Decoupled Weight Decay (AdamW)

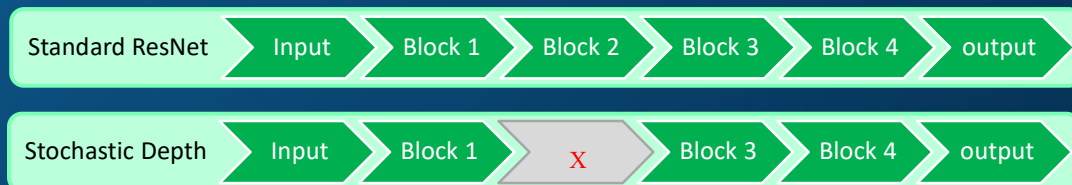
5/8/2026

pra-sâmi

47

Stochastic Depth – For Very Deep Networks (ResNet) [Huang et al. 2016]

- ❑ Stochastic depth is a regularization technique specifically designed for very deep residual networks like ResNet
- ❑ Many layers in deep ResNets contribute very little — they learn to approximate identity.
 - ❖ Overfitting: More parameters than necessary for the task
 - ❖ Diminishing returns: Extra layers learn near-identity mappings (wasteful)
 - ❖ Long training time: All layers must be forward/backward propagated
 - ❖ Feature reuse: Many layers essentially bypassed via shortcuts anyway
- ❑ Randomly drop entire residual blocks during training



5/8/2026

pra-sami

48

Reflect...

- ❑ What is the primary purpose of Batch Normalization in deep learning?
 - ❖ A. To prevent overfitting
 - ❖ B. To reduce the number of parameters in the model
 - ❖ C. To accelerate training and reduce internal covariate shift
 - ❖ D. To increase the depth of the neural network
- ❑ Answer: C. To accelerate training and reduce internal covariate shift
- ❑ At which stage is Batch Normalization applied in a neural network?
 - ❖ A. After the input layer
 - ❖ B. After the activation function
 - ❖ C. Before the loss calculation
 - ❖ D. Before or after the activation function, depending on the implementation
- ❑ Answer: D. Before or after the activation function, depending on the implementation

5/8/2026

pra-sami

49

Reflect...

- ❑ Which of the following is a key step in Batch Normalization?
 - ❖ A) Normalizing the gradient updates
 - ❖ B) Normalizing the activations by subtracting the batch mean and dividing by the batch standard deviation
 - ❖ C) Initializing weights to zero
 - ❖ D) Adding noise to the input data
- ❑ Answer: B) Normalizing the activations by subtracting the batch mean and dividing by the batch standard deviation
- ❑ What are the two learnable parameters introduced in Batch Normalization?
 - ❖ A) Gamma and Beta
 - ❖ B) Alpha and Beta
 - ❖ C) Theta and Gamma
 - ❖ D) Sigma and Mu
- ❑ Answer: A) Gamma and Beta

5/8/2026

pra-sâmi

50

Reflect...

- ❑ How does batch normalization help in reducing the vanishing gradient problem?
 - ❖ A. By adding noise to the weights
 - ❖ B. By increasing model size
 - ❖ C. By maintaining stable input distributions across layers
 - ❖ D. By removing nonlinearities from the model
- ❑ Answer: C. By maintaining stable input distributions across layers
- ❑ Which of the following best describes the difference between batch normalization and layer normalization?
 - ❖ A. Batch normalization normalizes across neurons in a layer; layer normalization across the batch
 - ❖ B. Layer normalization is used for image data only
 - ❖ C. Batch normalization is used during inference; layer normalization is not
 - ❖ D. Layer normalization normalizes across features within a single data sample
- ❑ Answer: D. Layer normalization normalizes across features within a single data sample

5/8/2026

pra-sâmi

51

Reflect...

- ❑ What is the purpose of dropout in deep neural networks?
 - ❖ A. To add noise to the input data
 - ❖ B. To randomly drop neurons during training to prevent overfitting
 - ❖ C. To increase the learning rate
 - ❖ D. To increase the model complexity
- ❑ Answer: B. To randomly drop neurons during training to prevent overfitting
- ❑ What is the primary purpose of regularization in deep neural networks?
 - ❖ A. To increase computational efficiency
 - ❖ B. To prevent overfitting
 - ❖ C. To speed up convergence during training
 - ❖ D. To increase the model's capacity
- ❑ Answer: B. To prevent overfitting

5/8/2026

pra-sâmi

52

Reflect...

- ❑ Which type of regularization adds a penalty term to the loss function based on the absolute values of the weights?
 - ❖ A. L1 Regularization
 - ❖ B. L2 Regularization
 - ❖ C. Dropout
 - ❖ D. Batch Normalization
- ❑ Answer: A. L1 Regularization
- ❑ How does dropout regularization work?
 - ❖ A. It penalizes large weights in the network
 - ❖ B. It introduces noise to the input data during training
 - ❖ C. It randomly removes neurons during training
 - ❖ D. It normalizes the input features
- ❑ Answer: C. It randomly removes neurons during training

5/8/2026

pra-sâmi

53

Reflect...

❑ Which of the following statements best differentiates L1 from L2 regularization?

- ❖ A. L1 encourages sparsity, while L2 distributes weights evenly
- ❖ B. L1 uses squared weights, L2 uses absolute weights
- ❖ C. L1 increases model complexity, L2 reduces it
- ❖ D. L2 is useful only for binary classification tasks

❑ Answer: A. L1 encourages sparsity, while L2 distributes weights evenly

❑ The Frobenius norm of a matrix is defined as:

- ❖ A. The maximum value in the matrix
- ❖ B. The sum of absolute values of all entries
- ❖ C. The square root of the sum of squares of all entries
- ❖ D. The sum of eigenvalues of the matrix

❑ Answer: C. The square root of the sum of squares of all entries

5/8/2026

pra-sâmi

54

Reflect...

❑ Which regularization method penalizes the squared values of the weights in the network?

- ❖ A. Dropout
- ❖ B. L1 Regularization
- ❖ C. L2 Regularization
- ❖ D. Batch Normalization

❑ Answer: C. L2 Regularization

❑ What is the trade-off associated with increasing the strength of regularization in a deep neural network?

- ❖ A. Increased risk of overfitting
- ❖ B. Increased risk of underfitting
- ❖ C. Slower convergence during training
- ❖ D. Improved model generalization

❑ Answer: B. Increased risk of underfitting

5/8/2026

pra-sâmi

55

Reflect...

- ❑ Which regularization technique is commonly applied to prevent exploding gradients during training?
 - ❖ A. Dropout
 - ❖ B. Batch Normalization
 - ❖ C. L2 Regularization
 - ❖ D. Data Augmentation
- ❑ Answer: B. Batch Normalization
- ❑ What is the role of early stopping as a form of regularization?
 - ❖ A. To speed up the training process
 - ❖ B. To prevent the model from fitting the training data too closely
 - ❖ C. To add noise to the input data
 - ❖ D. To stop the training process when the model performance on a validation set plateaus or degrades
- ❑ Answer: D. To stop the training process when the model performance on a validation set plateaus or degrades

5/8/2026

pra-sâmi

56

Reflect...

- ❑ In the context of regularization, what does the term "lambda" typically represent?
 - ❖ A. Learning rate
 - ❖ B. Regularization strength
 - ❖ C. Number of hidden layers
 - ❖ D. Batch size
- ❑ Answer: B. Regularization strength
- ❑ Which regularization technique is particularly useful for handling sequences and time-series data in deep learning?
 - ❖ A. L1 Regularization
 - ❖ B. Data Augmentation
 - ❖ C. Recurrent Dropout
 - ❖ D. Batch Normalization
- ❑ Correct Answer: C. Recurrent Dropout

5/8/2026

pra-sâmi

57

Reflect...

- ❑ What is the purpose of data augmentation as a regularization technique?
 - ❖ A. To add noise to the input data
 - ❖ B. To increase the model's capacity
 - ❖ C. To generate more training samples by applying random transformations to the existing data
 - ❖ D. To decrease the learning rate during training
- ❑ Answer: C. To generate more training samples by applying random transformations to the existing data
- ❑ Which of the following is a common data augmentation technique in image classification tasks?
 - ❖ A. Normalizing pixel values
 - ❖ B. Random horizontal flipping
 - ❖ C. Feature selection
 - ❖ D. Label smoothing
- ❑ Answer: B. Random horizontal flipping

5/8/2026

pra-sâmi

58

Reflect...

- ❑ What happens if dropout is applied during inference (testing phase)?
 - ❖ A. It improves accuracy
 - ❖ B. It increases overfitting
 - ❖ C. It leads to inconsistent predictions
 - ❖ D. It helps generalize better
- ❑ Answer: C. It leads to inconsistent predictions

5/8/2026

pra-sâmi

59 Next Session...

- Vanishing Gradients
- Exploding Gradients
- Gradient Check

5/8/2026

pra-samī

60

THANK YOU

5/8/2026

pra-samī